

TRIE-AWARE TOOL MEMORY · FYP RESEARCH REPORT

Initial Research Findings

Task F deliverable: dataset inventory, schema-length and frequency figures, prefix-cache sanity results, ordering comparisons, and the recommendation for the next research stage.

- Local analysis and measured GPU results



CONTENTS

Result in one paragraph

Evidence obtained locally

Prefix-cache sanity results

Prefill cost and the measurement floor

Analytical estimate versus measured cache hits

Recommendation

Immediate cluster run

Executive answer

Public tool workloads contain a measurable analytical prefix-locality signal, but it is workload- and evidence-dependent. Deterministic ordering raises estimated block reuse over empirical order for both ToolRet and BFCL. Measured on vLLM, prefix caching yields no TTFT benefit at 303-token prompts but up to a 10.6x reduction once tool menus reach realistic, deployment-scale sizes, with an analytical reuse estimate that under-predicts measured cache hits by 1.2-1.5x. The recommendation is to proceed with the exact prompt-level ToolTrie baseline, focused on tool-specific cache admission rather than a generic reordering claim.

Result in one paragraph

The public workloads contain a measurable analytical prefix-locality signal, but it is workload- and evidence-dependent. Across 45,815 canonical tools, median schema length is 70 Qwen tokens and P95 is 220. ToolRet's median relevance set and BFCL's median exposed menu both contain one tool, which limits what reordering can achieve on many tasks. On empirical dataset order, schema-cost-weighted ordering raises the ToolRet tool-prefix block-reuse estimate from 26.91% to 31.80% (4.89 percentage points). Frequency ordering raises the BFCL menu estimate from 10.98% to 18.88% (7.90 points). These are analytical estimates, not vLLM hit-rate or latency results.

Evidence obtained locally

- ToolRet: 7,961 tasks, 7,652 tools appearing in gold relevance labels, with every label resolved.
- BFCL subset: 1,240 tasks and 1,362 distinct canonical functions appearing in exposed menus.
- Schema diagnostics flag 19,921/45,815 tools. Most flags are empty parameter objects, which can be legitimate; malformed and missing-field counts are reported separately.
- Controlled support-skewed replay produces much more reuse than uniform or empirical access, but under unbounded retention that gap comes from resampling with replacement — repeated requests — not from locality. Only the finite-cache tables in `access-patterns.md` speak to request ordering.
- Classic FP-tree global order equals the frequency order in this baseline. Conditional pattern mining is still untested.

Prefix-cache sanity results

Measured on Qwen/Qwen3-0.6B, block size 16, 11807 GPU blocks, 5 trials per scenario with the prefix cache reset before each trial. Intervals are 95% Student-t half-widths.

CHECK	CACHED / PROMPT TOKENS	REUSE	TTFT ON (MS)	TTFT OFF (MS)
Cold prompt	0 / 303	0.00%	52.4 ± 28.8	51.5 ± 27.3
Identical prompt reuse	288 / 303	95.05%	43.0 ± 1.2	40.5 ± 2.3
Changed second tool	128 / 309	41.42%	41.4 ± 1.1	42.3 ± 2.2
Reordered first two tools	48 / 303	15.84%	37.7 ± 5.7	43.0 ± 4.2
Original restored	288 / 303	95.05%	41.1 ± 2.1	39.4 ± 2.8

The control served `enable_prefix_caching=False` and reported 0 cached tokens in every scenario, so the cache-on/cache-off output comparison is meaningful.

Prefill cost and the measurement floor

Menus are one gold tool padded with distractors from a fixed global catalog. Cold is measured after a prefix-cache reset; warm is an identical repeat. 5 trials per point.

MENU TOOLS	PROMPT TOKENS	COLD TTFT ON (MS)	WARM TTFT ON (MS)	WARM TTFT OFF (MS)	SPEEDUP
1	250	25.6	23.4	23.1	1.0x
4	433	25.2	16.9	23.7	1.4x
16	1,771	62.1	35.3	56.5	1.6x
64	6,742	248.8	54.1	245.6	4.5x
128	13,422	647.8	85.2	640.7	7.5x
200	20,627	1282.7	120.7	1275.2	10.6x

Prefix caching first produces a material gain at **4 tools** and grows steeply from there.

The cache-disabled control shows cold and warm within noise of each other at every size, so the gap above is prefix caching and not a warmup artifact. Note that within-run cold-vs-warm separation alone is not sufficient evidence: the control also 'separates' at one tool, where no cache exists. The cache-on/cache-off comparison is the trustworthy one.

This supersedes the earlier reading that prefix reuse buys nothing. It buys nothing *at 303 tokens*, which is simply below the floor.

Analytical estimate versus measured cache hits

WORKLOAD	PREDICTED CACHEABLE	MEASURED CACHED	MEASURED/PREDICTED	PREDICTED REUSE	MEASURED REUSE
menu64-alphabetical	428,016	526,432	1.23x	38.42%	38.15%
menu64-frequency	40,448	60,864	1.50x	3.63%	4.41%

The tool-unit model under-predicts rather than over-predicts. It ignores the chat template and system preamble, which are identical across requests and are themselves cached, and that outweighs the partial blocks it loses at tool boundaries.

The ordering comparison inverts the gold-only recommendation. On padded menus, ordering by benchmark support puts the *task-specific* tools first and pushes the shared catalog behind them, destroying the common prefix. A stable global order that ignores task-specificity keeps the shared catalog in front. Frequency ordering is the right choice only when the whole menu is task-specific, which is not what a connected tool catalog looks like.

Recommendation

Proceed with the exact prompt-level ToolTrie baseline on the cluster, focusing on multi-tool tasks and session-local replay. Use schema-cost-weighted ordering as the strongest ToolRet candidate and frequency ordering as the strongest BFCL candidate from this first analysis. Report the original order and fixed-random controls alongside them.

The likely publishable refinement is not a generic “reorder context into a trie” claim, because closely related cache-aware context ordering already exists. The more defensible direction is tool-specific cache admission using schema cost plus workflow co-occurrence, while preserving an active/authorized tool manifest and measuring function-call quality.

Do not pursue arbitrary independent KV concatenation yet. First establish that native exact APC converts the local token-reuse signal into repeatable TTFT benefit without a BFCL quality regression. If it does not, narrow the project to characterizing the crossover regimes or pivot toward retrieval/menu reduction.

Immediate cluster run

1. Freeze model revision, tokenizer, chat template, vLLM version, GPU, block size, dtype, and server flags.
2. Run the cache-enabled and cache-disabled sanity probe.
3. Render and save complete prompt token IDs for each ordering.
4. Replay original, frequency, and schema-cost-weighted workloads with documented cold/warm policies and repeated trials.
5. Add BFCL name/argument/no-tool scores before interpreting latency.

TATM FYP · Public benchmark analysis · Generated from initial-findings.md